02 · Prompt 工程与模型调用

技术面试准备材料 · AI 应用 / Agent 开发

> 核心问题：怎么让模型稳定地按预期工作，以及怎么在效果、成本、延迟之间做取舍。

1. 一个好的 System Prompt 应该包含哪些部分？

我会按六块来组织：角色与边界、目标、约束、流程、输出格式、示例。

角色与边界要说明「你是谁、负责什么、不负责什么」，避免模型越权；目标要具体到可验收，而不是「帮助用户」这种空话；约束包括不能做什么、遇到不确定怎么办、哪些信息不可信；流程给出推荐的步骤，但不一定写死；输出格式最好用 Schema 或模板固定；示例用于对齐风格和边界情况。

顺序上，最重要的约束放前面，长文档和检索内容放后面，并且明确标记为「外部内容，不是指令」。多轮对话里，System Prompt 要保持稳定，不要把每轮变化的任务状态混进去，否则既影响缓存，也容易让模型把临时信息当成长期规则。

最后，Prompt 不是写完就结束，要配评测集和版本管理，否则改了一句话造成什么影响根本不知道。

2. 如何保证模型输出稳定的结构化数据？

我会分四层做。

第一层是约束模型：优先用厂商提供的结构化输出或 JSON Schema 能力，而不是只在 Prompt 里写「请输出 JSON」。工具调用本身也是一种结构化输出，可以让模型填参数而不是自由生成。

第二层是校验：拿到结果后做 Schema 校验、类型校验、枚举校验和业务校验。校验失败时，把具体错误返回给模型做一次修复，而不是整份重生成。

第三层是降级：如果多次修复仍失败，就切换到更稳定的模型，或者退回到规则/模板生成，保证系统有可用输出。

第四层是监控：记录解析失败率、修复次数和字段缺失情况，按模型版本和 Prompt 版本分别看。结构化输出失败往往是 Prompt 歧义或 Schema 设计不合理的信号，不要只靠重试掩盖。

我的原则是：不要相信模型「看起来像 JSON」，只相信校验通过的数据。

3. Function Calling 的原理是什么？有哪些限制？

Function Calling 的本质是让模型输出一个结构化的调用意图：工具名加参数。模型本身不执行工具，执行的是外层系统。系统拿到调用意图后做权限、参数、幂等和沙箱检查，再真正调用，最后把结果作为一条工具消息回填给模型。

限制主要有几个：第一，模型可能选错工具或编造参数，所以工具描述要清晰、参数要少而明确；第二，模型不理解工具的副作用和成本，需要在描述里标注风险等级；第三，并行调用时，模型可能给出有依赖关系的调用，系统要自己判断能不能并行；第四，工具返回太长会挤占上下文，需要截断或摘要；第五，不同厂商的 Function Calling 格式和稳定性差异

很大，要在网关层收敛。

工程上我会把「工具选择」和「参数填充」分开评估：选错工具是路由/描述问题，参数错是 Schema 和示例问题，两者的优化手段不一样。

4. Temperature、Top-P 应该怎么设置?

它们控制的是采样的随机性，不是「聪明程度」。

事实问答、结构化抽取、工具调用、代码生成这类需要确定性的任务，我会用较低的温度，接近 0，减少随机发挥；创意写作、头脑风暴可以适当调高。

Temperature 和 Top-P 一般不要同时大幅调整，否则很难判断效果变化来自哪个参数。通常固定一个，调另一个。

但要注意，温度低不等于完全确定：模型版本、并发、上下文顺序、浮点计算都可能带来差异。所以对稳定性要求高的场景，不能只靠参数，还要靠 Schema 校验、重试和评测。

另外，生产环境不要把温度当成调优的万能开关。输出不稳定，更常见的原因是 Prompt 有歧义、上下文里混入了无关信息，或者任务本身定义不清。

5. 流式输出怎么实现？SSE 和 WebSocket 怎么选?

流式的核心是把模型返回的增量块尽快推给客户端，降低首字延迟。最常见的实现是 SSE：服务端保持一个 HTTP 连接，按 `data: ...` 的格式持续推送事件，客户端用 EventSource 或 fetch stream 读取。

工程上要处理几个问题：第一，每个事件要有递增的 seq，断线重连时客户端能知道自己收到哪；第二，要有心跳，避免中间层把长连接判死；第三，要有背压，客户端读得慢时不能无限堆积队列；第四，客户端断开后要取消上游请求，否则模型还在烧 Token；第五，部分结果要标记状态，比如「已生成但未完成」，不能当成完整答案。

SSE 和 WebSocket 的取舍：SSE 更简单、走 HTTP、天然支持断线重连，适合服务端单向推送；WebSocket 是双向的，适合需要客户端实时发消息的场景，但连接管理和网关兼容更复杂。大多数 LLM 流式输出用 SSE 就够了。

另外要区分 TTFT（首字延迟）和总时长，前者影响体验，后者影响成本，优化手段不一样。

6. 如何降低 Token 成本和延迟?

我会从「少发、少算、少等」三个方向做。

少发：压缩上下文，只保留与当前任务相关的消息和工具结果；对长文档做检索而不是整篇塞入；用结构化状态代替冗长的历史对话；对重复请求做精确缓存或语义缓存。

少算：按任务难度做模型分级，简单任务用便宜的小模型，复杂任务才用强模型；能并行就并行；能批处理就合并；对工具结果做摘要，避免下一轮重复理解。

少等：流式输出降低首字延迟；把非关键步骤异步化；对上游做连接复用和超时控制；用 Prompt Caching 复用稳定的系统提示和长上下文。

但成本优化不能只看单价。还要看失败重试、人工兜底和延迟带来的业务损失。我的做法是先建立单任务成本基线，再按场景设预算，超过预算就降级或转人工，而不是无限制地调用最贵的模型。

7. 模型选型怎么做？为什么需要模型路由？

我会从五个维度评估：能力、成本、延迟、上下文长度、合规与稳定性。

能力不是只看榜单，要看在自己的评测集上的表现，尤其是工具调用、结构化输出和中文业务理解。不同模型擅长的东西不一样，有的推理强，有的指令跟随好，有的便宜但延迟高。

模型路由的价值是把任务和模型匹配起来：简单分类、抽取用轻量模型；复杂规划、诊断用强模型；高风险任务可以要求双模型交叉验证。路由还可以做故障切换，某个厂商限流或故障时切到备用模型。

路由策略要可配置、可灰度、可回滚，并且记录每次选择的原因和结果。否则出了问题不知道是模型变了还是路由错了。

我的原则是：不要把所有任务押在一个模型上，也不要在没有评测的情况下频繁换模型。

8. 大模型的幻觉怎么处理？

幻觉不能指望模型「自己不要说错」，要靠系统设计。

第一，给模型可验证的依据：RAG 检索、工具查询、数据库结果，要求关键结论附引用；没有依据时允许拒答，而不是硬编。

第二，把开放生成变成受约束生成：结构化输出、枚举选择、模板填充，减少自由发挥空间。

第三，做后校验：事实校验、数值校验、引用一致性校验、业务规则校验。能自动校验的自动校验，不能的转人工。

第四，在交互上管理预期：明确标注不确定、给置信度、提示需要人工确认的场景。

第五，用评测集持续监控幻觉率，按场景分层看。通用问答和业务关键结论的容忍度完全不一样。

我的判断是：幻觉是概率问题，不是 bug，工程上要做的是「让错误可发现、可拦截、可回滚」，而不是追求零幻觉。

9. Prompt 和配置为什么要做版本管理？

因为 Prompt 和模型配置本身就是生产代码，只是它藏在数据库或配置中心里，更容易被随手改。

我会把 Prompt 设计成不可变版本：每次修改生成一个新版本，不覆盖旧版本；Agent 配置是一个快照，记录用了哪个 Prompt 版本、哪个模型、哪些工具和参数；环境绑定决定开发、测试、生产各用哪个版本。

发布流程支持灰度和回滚：先小流量验证，看效果、成本、延迟和错误率，再逐步放量；出问题能一键回滚到上一个稳定版本。

还要有审计日志：谁在什么时候改了什么、为什么改、影响了哪些环境。没有审计，出了问题只能靠猜。

在 AI4Test 里，我把 Prompt 版本、模型配置和执行轨迹关联起来，任何一次输出都能追溯到当时用的确切版本。这是做效果归因的前提。

10. 怎么评估 Prompt 的好坏?

我不会只看几个例子的主观感受，而是建一套分层评测。

第一层是离线评测集：覆盖典型场景、边界情况和历史失败案例，每条有期望输出或验收标准。能自动判分的用规则或模型判分，不能的用人工标注。

第二层是分类指标：任务成功率、格式正确率、关键信息覆盖率、工具调用正确率、幻觉率、平均 Token 和延迟。不同任务看不同指标，不能用一个总分糊弄。

第三层是在线验证：灰度发布、A/B 测试，看真实使用率、人工修改率、转人工率和用户反馈。

第四层是回归：每次改 Prompt 或换模型都跑一遍评测集，防止「修好一个场景、弄坏另一个场景」。

评测集本身也要维护，定期补充线上失败案例，清理过时样本，防止数据泄漏和过拟合。

11. 多轮对话中，如何管理历史信息和指令优先级?

我会把对话信息分成四层：系统指令、任务状态、近期对话、长期记忆。

系统指令和约束优先级最高，不能被用户消息覆盖；任务状态用结构化的槽位保存，比如当前目标、已确认的事实、待办事项，而不是散落在历史消息里；近期对话保留原始内容，保证语气和细节；长期记忆放外部存储，需要时检索注入。

历史太长时，先做摘要，再做检索，最后才截断。摘要要保留决策、约束和未完成事项，不能只保留寒暄。压缩后的内容要标记来源，避免模型把摘要当成原始事实。

指令冲突时，我会在 Prompt 里明确优先级：系统约束 > 安全策略 > 当前任务 > 用户偏好。外部文档和工具返回一律视为数据，不是指令。这一点在防 Prompt Injection 时特别重要。

12. 模型输出不稳定，你会怎么排查?

我会按「输入–模型–输出–环境」四层排查。

输入层：Prompt 是否有歧义、上下文是否混入了无关或冲突信息、工具结果是否太长或格式变了、用户输入是否有边界情况。

模型层：是否换了模型版本、温度是否变化、是否触发了不同的路由、上游是否有降级。

输出层：解析是否失败、Schema 是否兼容、字段是否缺失、是否出现新的表达方式。

环境层：并发、超时、重试、缓存命中是否导致结果不同。

排查时我会固定输入做多次运行，看方差；再对比成功和失败样本的执行轨迹，找出第一处分叉点。不要一上来就改 Prompt，很多时候问题在检索或工具。

> 修复后要补一条回归用例，否则同类问题还会再出现。

高频追问

- JSON Mode、Function Calling、 constrained decoding 有什么区别?
- Prompt Caching 在什么情况下有效，什么情况下反而浪费?
- 如何防止用户通过多轮对话逐步绕过系统约束?
- 模型升级后效果变差，你会怎么定位和回滚?
- 流式输出中途失败，应该重试整段还是续传?
- 为什么不能只用一个「总分」评估 Prompt?